Aula 11 — Ordenação e pesquisa

Organizando vetores e comparando pesquisa sequencial e binária

Dieimes Nunes

Retomando a Aula 10

Na aula anterior, aprofundamos a manipulação de listas.

Trabalhamos com:

- percorrer uma lista;
- somar valores;
- calcular média;
- encontrar maior e menor valor;
- contar elementos que atendem a uma condição;
- pesquisa sequencial;
- listas relacionadas.

Exemplo de pesquisa sequencial:

```
nomes = ["Ana", "Bruno", "Carlos", "Daniela"]
procurado = input("Nome: ")

encontrado = False

for nome in nomes:
    if nome == procurado:
        encontrado = True
        break

if encontrado:
    print("Nome encontrado")
else:
    print("Nome não encontrado")
```

Hoje vamos estudar uma pergunta importante:

> E quando precisamos organizar os dados antes de pesquisar?

O problema da aula

Imagine uma lista de notas:

```
notas = [8.5, 5.0, 9.2, 6.8, 7.5]
```

Ela está na ordem em que os valores foram cadastrados.

Mas podemos querer:

- mostrar da menor para a maior;
- mostrar da maior para a menor;
- localizar um valor com mais eficiência;
- descobrir a posição de um item;
- preparar os dados para uma pesquisa binária.

Para isso, precisamos compreender **ordenação** e **pesquisa**.

O que significa ordenar?

Ordenar significa reorganizar os elementos seguindo algum critério.

Exemplos:

```
5, 2, 9, 1, 7
```

Ordem crescente:

```
1, 2, 5, 7, 9
```

Ordem decrescente:

```
9, 7, 5, 2, 1
```

Também podemos ordenar textos alfabeticamente.

Atividade 1 — Ordenando no caderno

Sem usar o computador, coloque os valores abaixo em ordem crescente:

```
12, 5, 18, 3, 9, 15, 1
```

Depois, coloque em ordem decrescente.

Agora faça o mesmo com os nomes:

```
Carlos, Ana, Eduardo, Beatriz, Diego
```

Responda:

1. Qual critério você utilizou?
2. Foi necessário comparar elementos?
3. Você precisou trocar elementos de posição?
4. Um computador poderia seguir passos semelhantes?

Ordenação pronta no Python

Python possui ferramentas prontas para ordenar listas.

Podemos usar o método `sort()`:

```
numeros = [12, 5, 18, 3, 9]

numeros.sort()

print(numeros)
```

Resultado:

```
[3, 5, 9, 12, 18]
```

O método altera a própria lista.

Ordem decrescente

Para inverter a ordem:

```
numeros = [12, 5, 18, 3, 9]

numeros.sort(reverse=True)

print(numeros)
```

Resultado:

```
[18, 12, 9, 5, 3]
```

O argumento `reverse=True` solicita ordem inversa.

sort() e sorted()

Existem duas formas comuns.

sort()

Altera a lista original:

```
notas = [8, 6, 10, 7]
notas.sort()
print(notas)
```

sorted()

Cria uma nova sequência ordenada:

```
notas = [8, 6, 10, 7]
ordenadas = sorted(notas)

print(notas)
print(ordenadas)
```

Atividade 2 — Ordenação com Python

Crie um arquivo chamado:

`ordenacao_notas.py`

Faça o programa:

1. criar uma lista com pelo menos 6 notas;
2. mostrar a lista original;
3. criar uma nova lista em ordem crescente usando `sorted()`;
4. mostrar a ordem decrescente;
5. mostrar novamente a lista original.

Depois responda:

A lista original foi alterada quando usamos `sorted()`?

Mas como a ordenação funciona?

Usar `sort()` resolve o problema.

Porém, em Lógica de Programação, também é importante entender **como um algoritmo pode ordenar os valores**.

Existem vários métodos de ordenação.

Nesta aula, vamos estudar um dos mais didáticos:

> **Bubble Sort — ordenação por bolha.**

Ele não é escolhido por ser o método mais rápido, mas porque ajuda a visualizar comparações e trocas.

Ideia do Bubble Sort

Considere:

`5, 3, 8, 2`

Comparamos elementos vizinhos.

Primeira comparação:

`5 e 3`

Como `5 > 3`, trocamos:

`3, 5, 8, 2`

Depois comparamos `5` e `8`.

Não precisamos trocar.

Em seguida comparamos `8` e `2` e fazemos a troca.

Primeira passagem

Lista inicial:

`5, 3, 8, 2`

Comparando `5` e `3`:

`3, 5, 8, 2`

Comparando `5` e `8`:

`3, 5, 8, 2`

Comparando `8` e `2`:

`3, 5, 2, 8`

Observe:

> O maior valor chegou ao final da lista.

Atividade 3 — Bubble Sort manual

Faça no caderno uma passagem do Bubble Sort para:

```
7, 4, 9, 2, 6
```

Para cada comparação, registre:

```
Valores comparados: _____ e _____
Houve troca? Sim / Não
Lista após a comparação: ____________________
```

Depois responda:

> Qual valor terminou na última posição depois da primeira passagem?

Troca de valores em Python

Para trocar duas posições, podemos usar:

```
numeros = [5, 3]

numeros[0], numeros[1] = numeros[1], numeros[0]

print(numeros)
```

Resultado:

```
[3, 5]
```

Essa operação será usada no Bubble Sort.

Bubble Sort em Python

Uma versão didática:

```
numeros = [5, 3, 8, 2]

for i in range(len(numeros)):
    for j in range(len(numeros) - 1):
        if numeros[j] > numeros[j + 1]:
            numeros[j], numeros[j + 1] = numeros[j + 1], numeros[j]

print(numeros)
```

Resultado:

```
[2, 3, 5, 8]
```

Temos um `for` dentro de outro `for`.

Entendendo os índices

Nesta comparação:

```
if numeros[j] > numeros[j + 1]:
```

Estamos comparando:

```
posição atual        → j
próxima posição      → j + 1
```

Exemplo, se `j = 1`:

```
numeros[1]
numeros[2]
```

Por isso o laço interno não pode chegar até o último índice: não existiria uma posição seguinte.

Melhorando o Bubble Sort

A cada passagem, um dos maiores valores já fica na posição correta.

Podemos reduzir comparações desnecessárias:

```
numeros = [5, 3, 8, 2]

for i in range(len(numeros) - 1):
    for j in range(len(numeros) - 1 - i):
        if numeros[j] > numeros[j + 1]:
            numeros[j], numeros[j + 1] = numeros[j + 1], numeros[j]

print(numeros)
```

O `- i` evita revisar posições que já estão organizadas no final.

Atividade 4 — Implementando Bubble Sort

Crie:

`bubble_sort.py`

O programa deve:

1. criar uma lista com 8 números fora de ordem;
2. mostrar a lista original;
3. ordenar usando o algoritmo Bubble Sort;
4. mostrar a lista ordenada.

Não use `sort()` nem `sorted()` na etapa de ordenação.

Depois compare o resultado com:

```
print(sorted(numeros))
```

Pesquisa sequencial: revisão

Na pesquisa sequencial, verificamos um elemento de cada vez.

Exemplo:

```
numeros = [10, 25, 3, 18, 40]
procurado = 18

for i in range(len(numeros)):
    if numeros[i] == procurado:
        print(f"Encontrado na posição {i}")
        break
```

A pesquisa funciona mesmo se a lista estiver desordenada.

Quando a pesquisa sequencial termina?

Ela pode encontrar o valor logo no começo:

```
[18, 10, 20, 30, 40]
```

Ou somente no final:

```
[10, 20, 30, 40, 18]
```

Ou percorrer toda a lista e descobrir que o valor não existe.

Por isso, a quantidade de comparações pode variar.

Atividade 5 — Contando comparações

Considere:

```
[12, 7, 25, 18, 30, 5, 22]
```

Usando pesquisa sequencial, quantas comparações são necessárias para encontrar:

1. `12`?
2. `18`?
3. `22`?
4. `99`, que não está na lista?

Pesquisa binária

A pesquisa binária usa outra estratégia.

Em vez de verificar todos os elementos, ela compara com o **elemento do meio**.

Mas existe uma regra fundamental:

> A lista precisa estar ordenada.

Exemplo:

```
[3, 7, 10, 15, 18, 22, 30]
```

Queremos encontrar `18`.

Ideia da pesquisa binária

Lista:

```
[3, 7, 10, 15, 18, 22, 30]
```

Elemento central:

```
15
```

Queremos 18.

Como:

```
18 > 15
```

Podemos ignorar toda a metade esquerda.

Continuamos apenas com:

```
[18, 22, 30]
```

Dividindo o problema

Pesquisa binária repete a ideia:

1. encontrar a posição do meio;
2. comparar com o valor procurado;
3. se for igual, terminar;
4. se o procurado for menor, buscar à esquerda;
5. se for maior, buscar à direita.

A cada etapa, uma parte grande da lista é descartada.

Atividade 6 — Pesquisa binária no papel

Use a lista:

```
[2, 5, 8, 12, 15, 19, 23, 28, 31]
```

Procure manualmente o valor 23.

Registre em cada etapa:

```
Início: _____
Fim: _____
Meio: _____
Valor do meio: _____
Decisão: esquerda / direita / encontrado
```

Depois repita para o valor 5.

Índices usados na pesquisa binária

Precisamos controlar três posições:

```
inicio = 0
fim = len(numeros) - 1
meio = (inicio + fim) // 2
```

Exemplo:

```
0   1   2   3   4   5   6
3   7  10  15  18  22  30
```

Com `inicio = 0` e `fim = 6`:

```
meio = 3
```

O valor na posição 3 é `15`.

Pesquisa binária em Python

```
numeros = [3, 7, 10, 15, 18, 22, 30]
procurado = int(input("Valor procurado: "))

inicio = 0
fim = len(numeros) - 1
encontrado = False

while inicio <= fim:
    meio = (inicio + fim) // 2

    if numeros[meio] == procurado:
        encontrado = True
        print(f"Encontrado na posição {meio}")
        break
    elif procurado < numeros[meio]:
        fim = meio - 1
    else:
        inicio = meio + 1
```

Entendendo a atualização dos limites

Se o valor procurado for menor:

```
fim = meio - 1
```

Ignoramos o meio e tudo que está à direita.

Se for maior:

```
inicio = meio + 1
```

Ignoramos o meio e tudo que está à esquerda.

Isso diminui a área de busca.

Atividade 7 — Pesquisa binária em Python

Crie:

`pesquisa_binaria.py`

O programa deve:

1. possuir uma lista com pelo menos 10 números;
2. ordenar a lista antes da pesquisa;
3. solicitar um número ao usuário;
4. realizar a pesquisa binária;
5. informar se encontrou;
6. mostrar a posição quando encontrado.

Teste com:

Sequencial ou binária?

Pesquisa sequencial

- funciona com lista ordenada ou desordenada;
- é simples de implementar;
- percorre os elementos um a um.

Pesquisa binária

- exige dados ordenados;
- descarta metade da área de busca a cada etapa;
- exige controle de início, meio e fim.

Não existe um algoritmo “mágico” para todo problema.

Precisamos analisar o contexto.

Atividade 8 — Qual pesquisa escolher?

Para cada situação, indique **sequencial** ou **binária** e justifique.

Situação A

Uma lista com nomes cadastrados na ordem de chegada e sem ordenação.

Situação B

Uma lista de 10.000 códigos numéricos já ordenados.

Situação C

Uma lista pequena com apenas 5 produtos.

Situação D

Uma lista ordenada de matrículas escolares.

Não basta escolher: explique a decisão.

Cuidado: ordenar pode alterar a posição original

Imagine:

```
nomes = ["Carlos", "Ana", "Bruno"]
notas = [7.0, 9.0, 8.0]
```

As listas estão relacionadas pela posição.

Se ordenarmos apenas `nomes`:

```
nomes.sort()
```

A relação entre nome e nota será quebrada.

Esse é um problema importante.

Nas próximas aulas veremos estruturas melhores para representar registros completos.

Atividade 9 — Analisando um erro lógico

Considere:

```
nomes = ["Carlos", "Ana", "Bruno"]
notas = [7.0, 9.0, 8.0]

nomes.sort()

for i in range(len(nomes)):
    print(nomes[i], notas[i])
```

Responda:

1. O programa apresenta erro de sintaxe?
2. Ele executa?
3. Os dados continuam corretos?
4. Que tipo de erro aconteceu?

Esse é um exemplo de **erro lógico**: o programa executa, mas produz informações incorretas.

Onde estamos no curso

Até aqui, já conseguimos:

- criar algoritmos;
- armazenar dados;
- fazer cálculos;
- tomar decisões;
- repetir comandos;
- trabalhar com listas;
- pesquisar elementos;
- ordenar vetores.

O próximo passo será organizar dados em **linhas e colunas**, usando matrizes.

Desafio final da aula

Crie:

`sistema_numeros.py`

Menu:

```
1 - Cadastrar números
2 - Mostrar números
3 - Ordenar crescente
4 - Ordenar decrescente
5 - Pesquisa sequencial
6 - Pesquisa binária
0 - Sair
```

Regras:

- os números devem ficar em uma lista;
- a pesquisa sequencial pode funcionar em qualquer ordem;
- antes da pesquisa binária, garanta que a lista esteja ordenada;
- mostre a posição quando o valor for encontrado;
- o menu deve repetir até a opção `0`.

Fechamento

Nesta aula, trabalhamos com:

- ordenação crescente e decrescente;
- `sort()` e `sorted()`;
- lógica do Bubble Sort;
- trocas de posições;
- pesquisa sequencial;
- pesquisa binária;
- importância da lista ordenada;
- erros lógicos relacionados à posição dos dados.

Na próxima aula, vamos avançar para **matrizes** e estruturas organizadas em linhas e colunas.